Llama 3.1 70B Inference Capacity Report

Throughput, Latency, and Power Across a 150-Point Load Sweep

Background

Every day, millions of people interact with AI agents like ChatGPT, Claude, Gemini, and open models like Llama in a simple, familiar pattern: they ask a question, the agent responds, and if one keeps the conversation going, each new question carries the growing weight of everything said before it. A short first question might be a handful of words. By the tenth exchange, the "input" the model must process includes the entire conversation so far, while the "output" is however long the next response needs to be, from a quick one-line answer to a lengthy, detailed explanation.

This everyday pattern (growing input, variable output, and requests arriving at whatever pace people happen to type) is exactly what a production AI service must handle well, at scale, without becoming slow or unpredictably expensive to run. This study characterizes how one such system, Llama 3.1 70B served on real GPU hardware, behaves under this kind of varying load: how prompt length, response length, and request volume together determine how fast the system responds, how much electrical power it draws, and how busy the hardware actually is.

Executive Summary

The report presents a comprehensive performance evaluation of Llama 3.1 70B FP8 conducted via vLLM, utilizing 4-way tensor on four NVIDIA RTX PRO 6000 Blackwell GPUs connected by PCIe (no NVLink). The benchmark consisted of a sweep, a matrix of 150 distinct load conditions, comprising five input lengths, six output lengths, and five request rates at 100 requests per condition.

1. **Output length is the main driver of consumption.** Elapsed time and power usage increased based on output response length. Elapsed time went up from 19 seconds for output 128-tokens to about 815 seconds for output 10,240-tokens, time per output token from 37.8 to 64.7 ms, and average power from 195 W to 304 W per GPU. Throughput peaked near 1,750 tokens/s for 1,000 to 2,000-token outputs and falls about 28% at 10,240 tokens.
2. **Memory-bound and much less compute-bound.** Though GPU engine was busy 67 to 99% of the time, the tensor pipeline responsible for core mathematical matrix-multiply-accumulate operation were active only about 6 to 15% of the time, and delivered compute was about 4 to 6% of an estimated dense FP8 peak. Memory activity at 24 to 56% was consistently higher than tensor activity. The workload was limited by memory traffic, inter-GPU communication and scheduling overhead, not by tensor-core operations.
3. **KV cache fills its pool at long contexts**. vLLM reserved 85 to 86 GiB of each 96 GB GPU at startup with about 17 GiB for weights and up to about 68 GiB for the KV cache pool. The pool filled as the input and output tokens of all active sequences accumulate and reached 100% for 1,024-token

inputs with 10,240-token outputs. That pool held about 78 sequences at the full 11,264-token length; beyond that, vLLM paused sequences until space freed up.

The test measurements point to a large amount of unused compute, large context window size causing memory-bound system limiting full use of headroom compute capacity. A rough estimate puts the ceiling near 10,600 tokens/s, about 7 times the measured average of 1706 tokens/s (Section 6).

1. Sweep Configuration

Table 1. Sweep configuration

Parameter	Value
Model	Llama 3.1 70B [7] (dense transformer); FP8 weights, BF16 KV cache
Serving engine	vLLM 0.30.0 [6], tensor-parallel size 4, `--gpu-memory-utilization 0.88`, `--max-model-len 16384`
GPUs	4 × NVIDIA RTX PRO 6000 Blackwell Server Edition (96 GB each), PCIe, no NVLink
Load point input lengths	128, 256, 384, 512, 1,024 tokens
Load point output lengths	128, 512, 1,024, 2,048, 5,120, 10,240 tokens
Load point request rates (RPS)	5, 10, 20, 50, 100 req/s
Total load points	150 (5 × 6 × 5), 100 requests each
Prompt content	Random prompts; each request's length drawn uniformly within ±30% of the nominal value (fixed seed)
Collection dates	25 to 26 September 2026

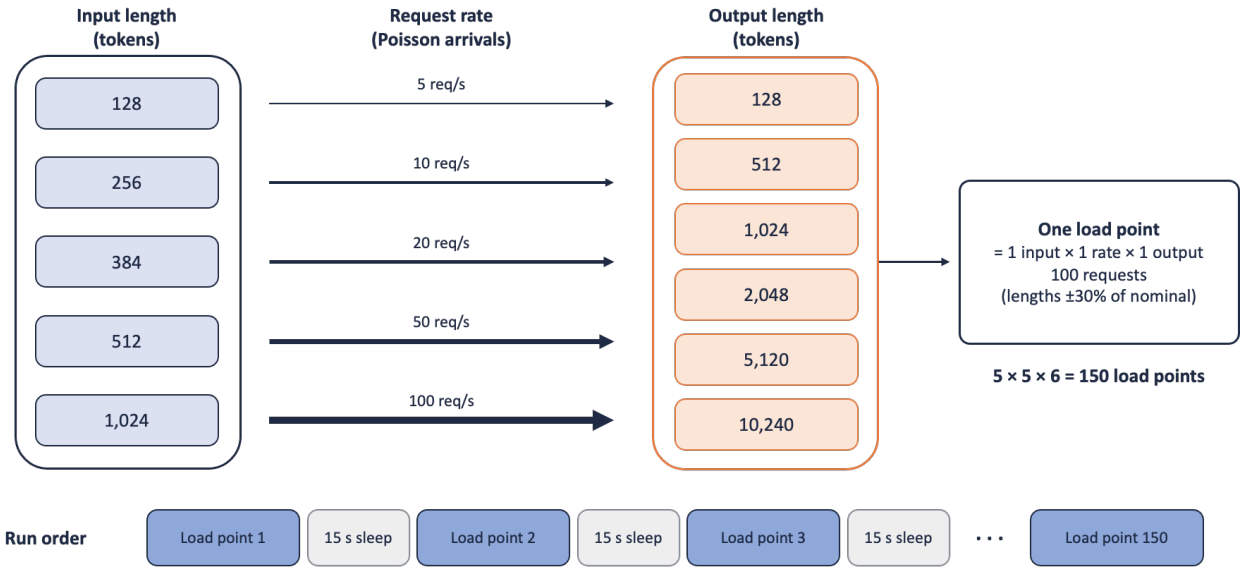


Figure 1. Sweep design. At 5 req/s the 100 requests arrive in about 20 s; at 100 req/s, within about 1 s. Each load point ran on its own, with a 15 s pause before the next so the GPUs and KV cache returned to idle.

Full methodology, including software setup and infrastructure debugging notes, are planned to be in a separate companion paper covering the benchmark environment and setup.

System architecture

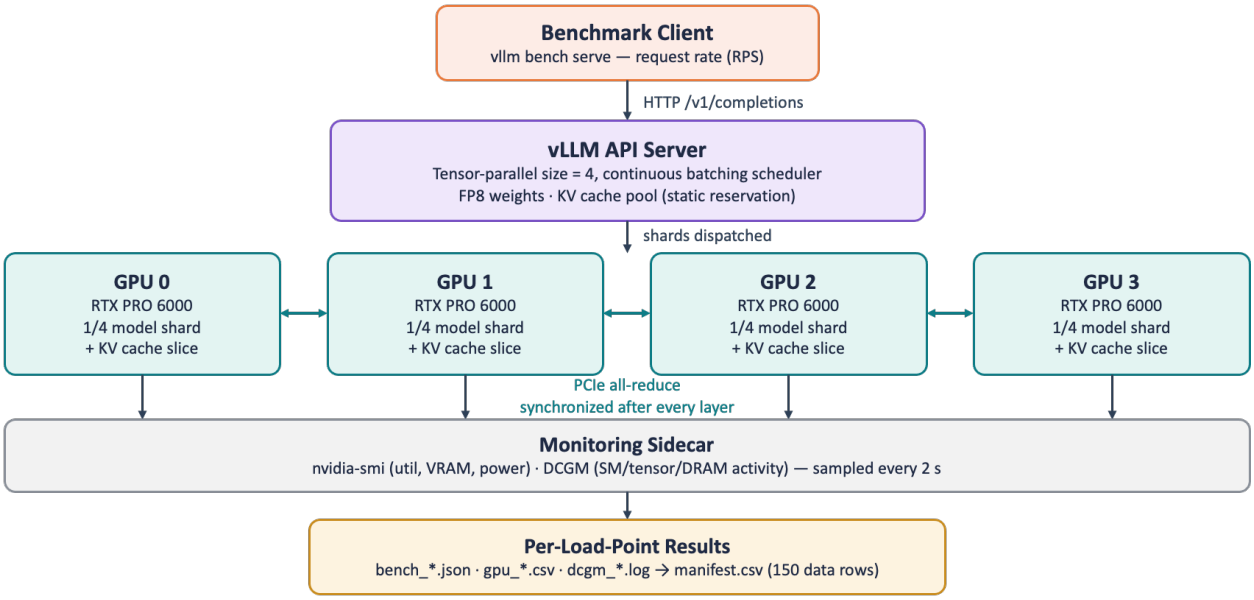


Figure 2. Benchmark architecture: client, vLLM server, four-GPU tensor-parallel layout, and monitoring pipeline.

Each request from the benchmark client is routed through vLLM's continuous batching scheduler [2, 6], sharded across all four GPUs by tensor parallelism, and synchronized after every model layer over PCIe. Its performance is sampled in parallel by a monitoring sidecar[1]. Results for each load point are captured separately and analyzed for use throughout this report.

2. Output (Response) Length Drives Consumption

The test results showed that output length increase from 128 tokens to 10,240 tokens showed that the resource consumption is driven by the length. Table 2 shows measurements for each output length, averaged across 25 load points (5 input lengths × 5 request rates). Time per output token (TPOT) is the mean time between successive generated tokens for a *single request*. Throughput tokens/s counts tokens from all requests in flight together. The product of them is the number of requests decoding at the same time:

`1202 tokens/s x 0.0378 s ≈` **`45 concurrent requests`**

[1]Implemented as two lightweight external processes sampling in parallel: `nvidia-smi` (utilization, VRAM, power draw) and NVIDIA's DCGM (`dcgmi dmon`, for SM, tensor and DRAM activity), each polling every 2 seconds, independent of the vLLM server process itself.

Table 2. Summary by output length (means over 25 load points)

Output length (tokens)	Elapsed time (s) for 100 requests	Throughput (tokens/s)	Time per output token (ms)	Avg power per GPU (W)	Tokens per joule
128	19	1,202	37.8	195	1.54
512	38	1,645	41.0	241	1.71
1,024	66	1,746	42.8	249	1.75
2,048	124	1,730	45.6	261	1.66
5,120	341	1,539	52.7	283	1.36
10,240	815	1,261	64.7	304	1.04

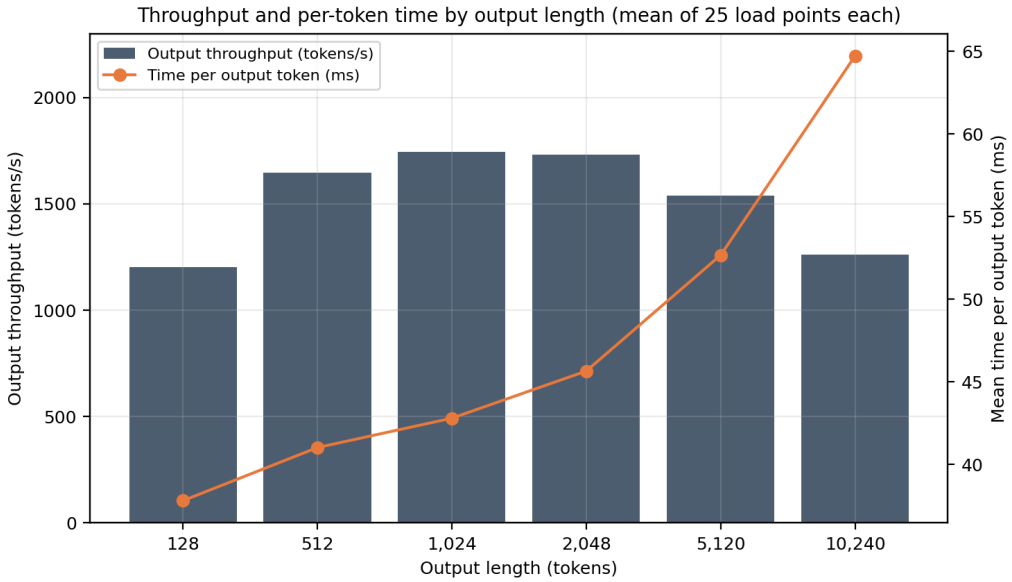


Figure 3. Throughput and time per output token by output length.

- **Elapsed time** grew in proportion to output length, slightly faster at the long end: a 5× increase from 2,048 to 10,240 tokens raised run time by 6.6× (124 s to 815 s), because per-token time also rises.
- **Throughput** was highest (about 1,750 tokens/s) for 1,024 to 2,048-token outputs. At 128 tokens output length the run was short enough that request arrival and start-up constituted a large part of it. At 10,240 tokens output length, per-token time was 71% higher, consistent with longer contexts requiring more KV cache reads at every step. By Little's law, concurrency is token throughput per second multiplied by time per token. During 10,240 token output length runs, based on 1261 tokens/s x 0.0647 s ≈ 82 sequences were concurrent.
- **Energy per token** peaked at about 1.75 tokens per joule for 1,024-token outputs and fell 41% to 1.04 at 10,240 tokens, because token throughput slowed down significantly in comparison to power usage. Power usage was derived as average across all four GPUs.
- **Input prompt length** had a less of an effect. For increase from 128 to 1,024 input tokens, time per output token rose 13% (45.4 to 51.2 ms), throughput fell 8% (1,569 to 1,442 tokens/s), average

power rose 3% (252 W to 260 W), and mean time to first token at 20 req/s rose from 90 ms to about 110 ms for outputs up to 2,048 tokens.

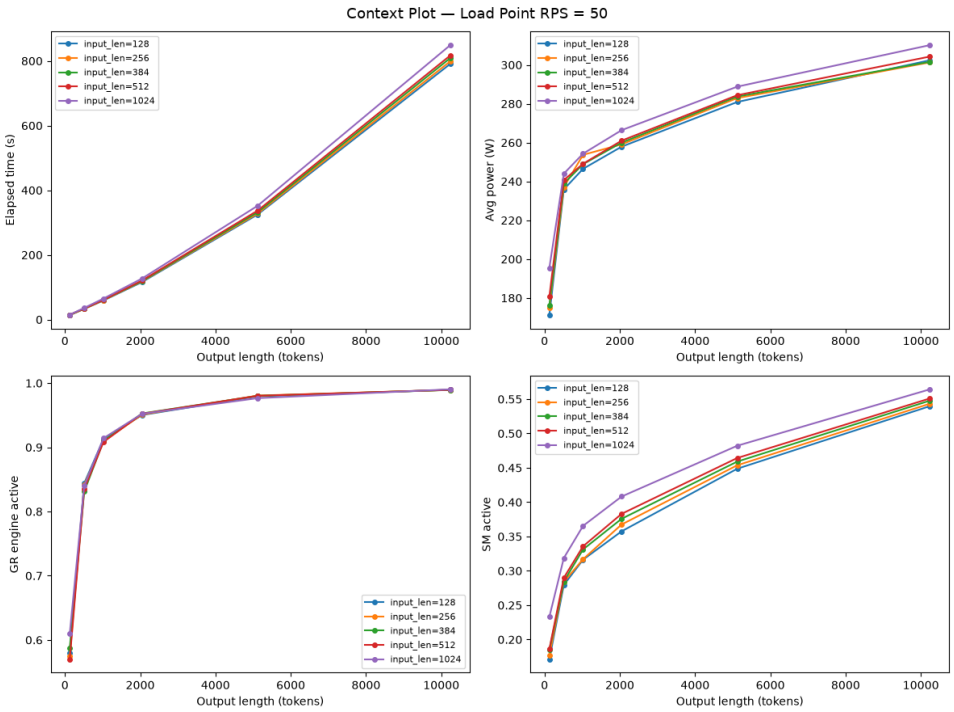


Figure 4. Run time, average power, GR engine activity and SM activity versus output length at 50 req/s, one line per input length. The pattern is similar at the other request rates; Section 3 shows where rate does matter.

Figure 3 shows that prompt-length lines sit close together, indicating only modest variation; the 1,024-token line stands apart most in SM activity. All four metrics rise with output length: run time almost linearly, power and SM activity steadily after an initial bend below about 1,000 tokens, while GR engine activity levels off near 0.95 to 0.99. The initial bend corresponds to the short (about 14 s) runs at 128-token outputs, where idle start-up time weighs most.

3. Concurrency and Sustainable Load

Each load point sent 100 requests and measured the server until all were served. The request rate represented requests arrival rate: in 20 s at 5 req/s and within about 1 s at 100 req/s. Once a run lasted more than about two minutes (outputs of 2,048 tokens or more), all 100 requests were in flight together at every rate, and the server held about 80 sequences at the same time. This section therefore states result in terms that carry over to any arrival pattern: sequences in flight, the request rate that the server can sustain, and time to first token when many requests arrive together.

Sustainable request rate. The request rate a server can keep up with indefinitely is its throughput divided by the tokens in each response (Table 3). This is the figure to use for capacity planning. The 5 to 100 req/s used in the sweep only set how quickly each batch of 100 requests was submitted.

Table 3. Sustainable request rate per server, by output length

Output length (tokens)	Throughput (tokens/s)	Sequences in flight (avg)	Sustainable rate (requests/s)	Requests per hour
128	1,202	47	9.4	33,800
512	1,645	68	3.2	11,600
1,024	1,746	75	1.7	6,100
2,048	1,730	79	0.84	3,000
5,120	1,539	81	0.30	1,100
10,240	1,261	82	0.12	440

Measured at about 50 to 80 sequences in flight; higher concurrency may raise these rates toward peak estimates in Section 6. The 128-token rate is understated because these short runs include about 5 to 6 s of tool start-up and teardown (Appendix A).

Per-token time and power. For outputs of 2,048 tokens or more, per-token time and power were similar (46 to 65 ms per token and 261 to 304 W per GPU) and remained so however fast the requests arrived.

Time to first token (TTFT) was 85 to 125 ms for 95% of the load points for all output lengths except 10,240-token outputs (Table 4). For 10,240-token outputs when many requests arrived together: mean TTFT was about 200 ms when the 100 requests arrived over about 20 s, while 800 to 1100 ms when they arrived within 1 to 2 s, and about 3 s for 1,024-token inputs (Table 5). In these runs the KV cache approached full capacity (Section 5), so new requests waited to be admitted. KV fill was about the same however fast the requests arrived, so the rise reflected many requests arriving at once, not a fuller cache. Per-token time is unaffected.

Table 4. Mean time to first token (ms) by output length and the time over which the 100 requests arrived (mean of the five input lengths)

Output length (tokens)	At ~20 s	At ~10 s	At ~5 s	At ~2 s	At ~1 s	Range over the 25 load points
128	133	90	97	104	108	85–274
512	96	97	98	103	109	90–125
1,024	97	96	98	103	109	90–124
2,048	157	96	97	103	108	90–296
5,120	95	95	96	102	107	88–124
10,240	197	178	338	821	1,130	101–2,959

The ~20 s means for 128- and 2,048-token outputs are raised by two single runs with 1,024-token inputs (274 ms and 296 ms); the other four input lengths at those settings are 96 to 125 ms.

Table 5. Mean time to first token (ms) for 10,240-token outputs, by input length and the time over which the 100 requests arrived

Input length (tokens)	At ~20 s	At ~10 s	At ~5 s	At ~2 s	At ~1 s	Peak KV cache fill (%, max over arrival times)
128	121	101	106	126	172	89.1
256	131	115	130	201	462	90.8
384	145	136	168	415	852	92.0
512	166	168	235	876	1,205	93.9
1,024	421	373	1,051	2,487	**2,959**	100.0

Peak KV cache fills up at largest load point. TTFT jumped up where the cache is closest to full; the worst load point (1,024-token inputs, arriving at ~1 s) is shown in bold.

4. GPU Usage Trend

GPU telemetry reports activity at several levels, from the whole engine down to the tensor pipes. Table 5 defines the metrics used here. All are time-averages over the run window across the four GPUs.

Table 4. GPU metrics (NVIDIA DCGM)

Metric	What it measures
GR engine active (GRACT)	Fraction of time the GPU's graphics/compute engine had any work. The coarsest signal; it saturates near 1.0.
SM active (SMACT)	Fraction of time the streaming multiprocessors had at least one resident warp (a group of 32 threads executed together), averaged over SMs.
SM occupancy (SMOCC)	Resident warps as a fraction of the maximum an SM can hold: how full the active SMs are.
Tensor active (TENSO)	Fraction of hardware cycles that the GPU's Tensor Core pipeline is actively executing instructions
DRAM active (DRAMA)	Fraction of cycles the memory interface was transferring data.
PCIe TX / RX	Bytes per second sent and received over PCIe per GPU. Includes all PCIe traffic, not only tensor-parallel all-reduce.

Table 5. GPU activity by output length (means over 25 load points)

Output length (tokens)	GR engine active	SM active	SM occupancy	Tensor active	DRAM active
128	0.667	0.248	0.022	0.075	0.237
512	0.851	0.312	0.026	0.105	0.311
1,024	0.906	0.342	0.028	0.113	0.345
2,048	0.952	0.383	0.030	0.123	0.388
5,120	0.973	0.459	0.034	0.135	0.466
10,240	0.990	0.549	0.039	0.150	0.555

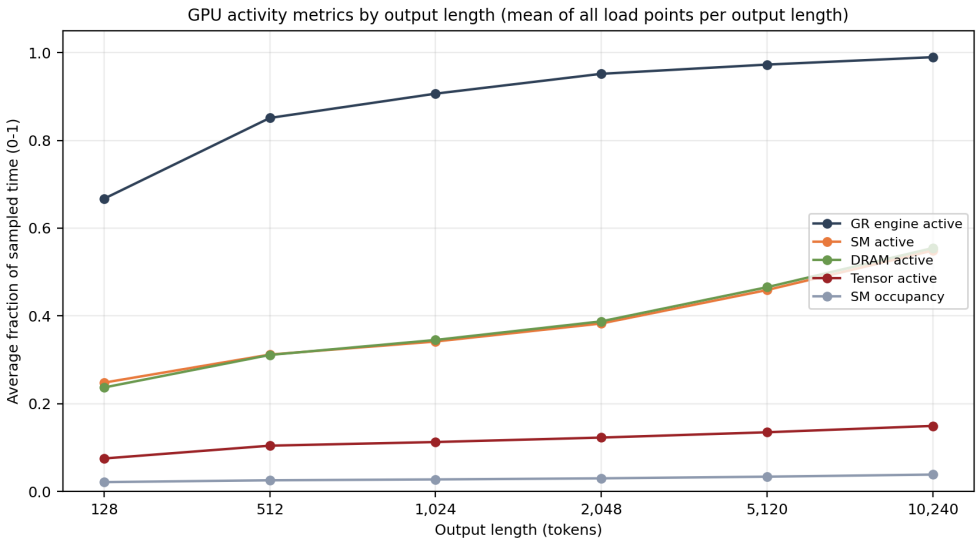


Figure 6. GPU activity metrics by output length.

- **GPUs active but far from full.** In Figure 6 GR engine activity rises from 0.67 to 0.99, yet SM occupancy is only 0.02 to 0.04 and tensor activity 0.06 to 0.15 on average (peak 0.245). The SMs are running, but with few warps resident and mostly not on tensor-core work.
- **GR engine activity saturates early; the others keep rising.** GR engine activity reaches 0.95 by 2,048 tokens and cannot rise much further. SM, tensor and DRAM activity keep climbing with output length (DRAM active more than doubles, 0.24 to 0.56), consistent with longer contexts adding attention and KV cache work to every decode step.
- **Memory activity exceeds tensor activity at every output length** (by 3 to 4 times), the sign of a memory-bandwidth-bound token generation phase [3]: tied up in streaming the model weights and the KV cache, while the computation remains low.
- **SM active and DRAM active track each other within about 1%**, and most of the additional time per token at long outputs corresponds to additional memory-busy time, consistent with a memory-bound decode phase.

Compute and inter-GPU traffic

Compute: At a requirement of about 140 GFLOP per generated token (2 × 70 billion parameters), the measured throughput corresponded to roughly 170 to 245 TFLOPS in total across the four GPUs, which is 4 to 6% of an estimated peak dense FP8 capacity of 1,000 TFLOPS per GPU. (NVIDIA publishes only a 2,000 TFLOPS FP8 figure that assumes 2:4 sparsity, which these dense weights do not use, so the dense peak of 1,000 TFLOPS is our estimate.) This agrees in magnitude with the 6 to 15% tensor activity. Delivered compute at 4 to 6% of the estimated dense FP8 peak, and tensor pipes activity factor (duty cycle) at 6 to 15% of the time. Both indicate that the tensor hardware is used well below its capacity. The delivered compute is lower than the duty cycle when tensor units are active but not running at peak rate.

Traffic: Tensor parallelism synchronizes the GPUs after every layer over PCIe [3]. Average transmit traffic was 5.9 to 13.4 GB/s per GPU (peaks to 31 GB/s). Related closely to throughput, about 6 MB was sent per generated token per GPU, at 5.8 to 6.1 MB for output lengths of 512 tokens and above. On an average, number of requests in flight was about 72 (47 for 128-token outputs, 82 for 10,240-token outputs. To dive deeper into traffic estimation, for 10,240-token outputs, 82 requests × 6.1 MB traffic is about 500.2 MB per token. A token takes 64.7 ms, which gives 7.7 GB/s, matching the measured value. For 1,024-token outputs, 75 × 5.8 MB is about 434 MB per 42.8 ms, which is 10.1 GB/s.

5. Memory: Role of KV Cache in Reserved Memory

Reserved VRAM was effectively constant at 84.9 to 86.2 GiB per GPU at all 150 load points. This was pool vLLM set aside at startup (`--gpu-memory-utilization 0.88`) [6], provisioning 16.98 GiB for model weights while setting aside up to about 68 GiB for KV cache pool. Longer sequences fill more of that pool but do not enlarge the reserved capacity [1], so VRAM reported by `nvidia-smi` or DCGM cannot be used as a load or capacity signal.

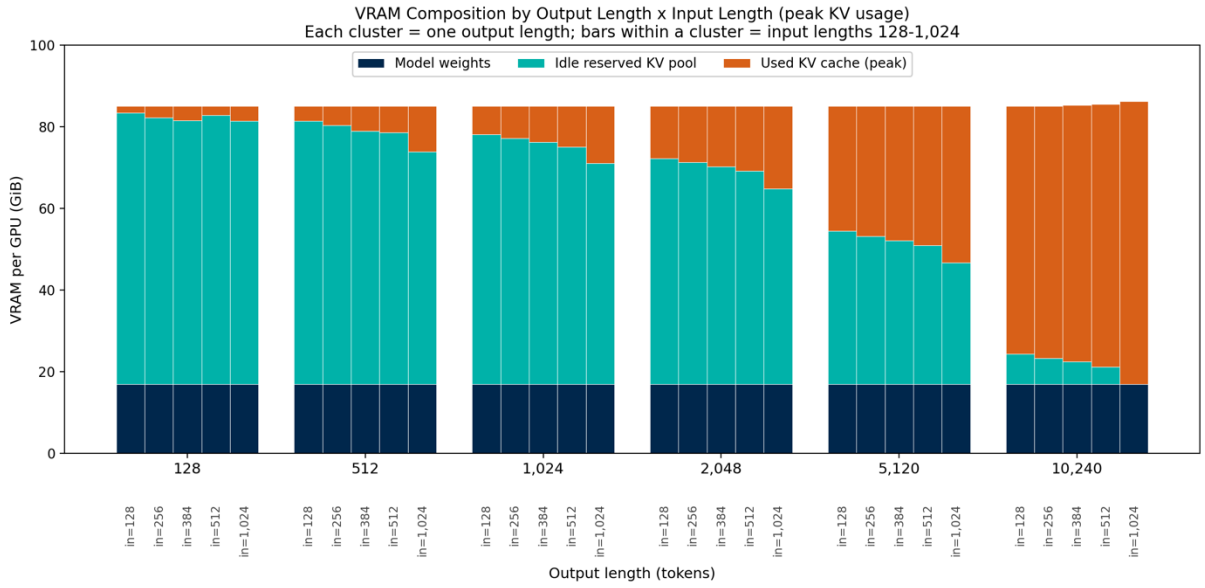


Figure 7. VRAM per GPU by output length; bars in each cluster run from 128 to 1,024 input tokens (peak KV use, maximum over the five rates).

vLLM does report how full the pool is [6]. The "GPU KV cache usage" has been parsed from the vLLM server logs after matching them to each load point (Table 6). KV Cache Pool fill rose steeply with output length, and for 10,240-token outputs it rose with input length from 89% to 100% of the pool.

Table 6. KV cache in use (percent of the pool; vLLM server logs)

Output length (tokens)	Average over run	Peak (mean of 5 input lengths)	Peak, 128-token inputs	Peak, 1,024-token inputs
128	1.8	2.7	2.5	5.5
512	5.2	8.6	5.5	16.4
1,024	8.3	13.5	10.2	20.7
2,048	13.8	22.5	18.8	29.7
5,120	30.2	48.7	44.9	56.4
10,240	58.6	92.8	89.1	100.0

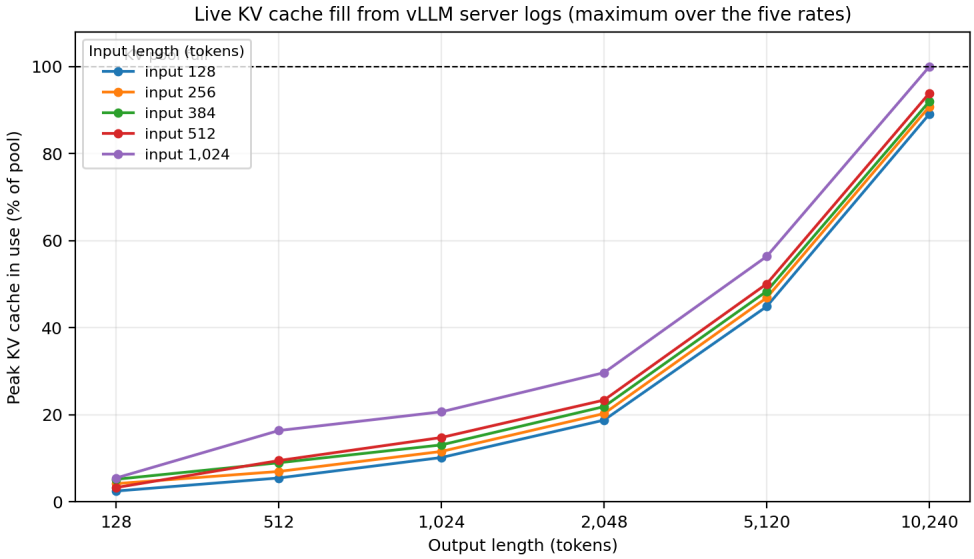


Figure 7. Peak KV cache fill by output length, one line per input length.

The cache holds about 80 KB per token per GPU (BF16, 8 KV heads split across 4 GPUs) [7]. Table 7 shows how many sequences of a given context length would fit. Holding 100 sequences of 10,000 tokens at the same moment would need more than the whole KV Cache pool; the measured peak stayed below 100% for most load points because the sequences were at different stages and their lengths varied around the nominal value.

Table 7. KV cache capacity by context length (upper bound; pool of about 68 GiB per GPU)

Context (input + output tokens)	KV cache per sequence per GPU (GiB)	Sequences that fit
1,152 (128 + 1,024)	0.088	776
5,248 (128 + 5,120)	0.400	170
10,368 (128 + 10,240)	0.791	86
11,264 (1,024 + 10,240)	0.859	79

6. Estimated Maximum Throughput

The measurements suggest a large amount of unused compute, but we did not test higher concurrency, so the following are estimates (Post Study Calcs.xlsx, Headroom sheet).

- **Compute.** At 140 GFLOP per token and an estimated 1,000 TFLOPS dense FP8 per GPU, four GPUs could in principle generate about 28,600 tokens/s at full tensor utilization, or 14,000 to 17,000 at 50 to 60%.
- **Batch size.** Each token-generation step produces one new token for every request in flight, and the GPUs stream the model weights from memory once per step. That alone takes about 11 ms per GPU (17 GiB at 1.6 TB/s) [3]. Compute time per step reaches the same 11 ms at roughly 330 concurrent sequences (the roofline model's *ridge point*); below that, the weight reads dominate and extra sequences add little time. The sweep held about 72 concurrent sequences on average.
- **PCIe.** Derived from `dcgmi dmon` counters and measured throughput, each GPU transmitted about 6 MB per generated token. Assuming a PCIe 5.0 x16 link (about 63 GB/s per direction per GPU), inter-GPU traffic alone would cap throughput at roughly 10,600 tokens/s, an upper bound that assumes the link runs at its theoretical peak.

Table 8. Estimated maximum throughput (4 GPUs)

Estimate	Tokens/s	Multiple of measured average
Measured average over all load points	1,521	1.0×
PCIe-limited (assumed 63 GB/s link, 5.96 MB per token)	10,574	7.0×
Compute-bound at 50% of estimated peak	14,286	9.4×
Compute-bound at 60% of estimated peak	17,143	11.3×
Compute-bound at 100% of estimated peak	28,571	18.8×

The PCIe-limited figure is the lowest, so inter-GPU communication, not tensor-core throughput, is the more likely ceiling for tensor-parallel decoding on these cards, and the realistic headroom is probably below 7×. Confirming it needs a run with many more concurrent requests (for example 500 to 1,000 at 512 to 1,024-token outputs) and a check of the actual PCIe link speed.

7. Practical Takeaways

1. **Plan capacity by output length.** The cost of a request, in time and energy, scales with the tokens it generates: run times in this sweep differ about 40-fold between the shortest and longest outputs, and tokens per joule fall 41% from the best case to 10,240-token outputs.
2. **Monitor time to first token and KV cache fill.** TTFT stays near 100 ms until the KV cache approaches full, then rises quickly.

3. **Size for concurrent requests, not only request rate.** Throughput is roughly the number of requests in flight divided by time per token. In this sweep about 47 to 82 requests were in flight on average, each receiving 15 to 26 tokens/s, for 1,200 to 1,750 tokens/s in aggregate. At long contexts, KV cache memory limits concurrency (roughly 80 to 170 sequences of 5,000 to 10,000 tokens per server, an upper bound), and time to first token rises sharply as the cache nears full. The sweep never exceeded 100 concurrent requests, so behavior above that that is untested.
4. **Precision matters**: Model precision dictates the boundary between compute-bound and memory-bound execution in the inference pipeline. While native 16-bit precision preserves maximum generation quality, transitioning to low-precision formats (e.g., FP8 or INT8) reduces the KV cache and weight footprint by 50% or more. This memory relief directly translates to higher throughput, allowing the serving engine to sustain larger concurrent batch sizes and higher request rates before reaching the saturation knee-point.
5. **Expect unused compute.** Tuning concurrency and batching may raise throughput substantially, but PCIe bandwidth could become a limiting factor. Test before relying on the numbers.

8. Limitations

- **Single test run per load point.** There is no variance estimate, so differences of a few percent cannot be distinguished from noise. One load point (256-token inputs, 1,024-token outputs, 20 req/s) shows a transient dip, with average utilization of 73% against about 90% at the other rates and a run time of 75 s against 59 to 73 s, consistent with a brief interruption. It is left in the data.
- **Concurrency.** Each load point sends 100 requests, so concurrency never exceeds 100. This limits what the sweep can say about higher load. The average of about 72 requests in flight is derived from throughput and time per token, not measured directly.
- **Measurement window.** Run time includes the time to submit the requests and a few seconds of tool start-up and teardown, and activity and power averages include idle samples at the edges. These dilute the averages for runs shorter than about two minutes.
- **Prefix caching.** Prompts come from a fixed seed, so a given input and output shape is expected to use the same 100 prompts at each of the five request rates. Automatic prefix caching was enabled, and the server log reported hit rates from 0 to 99% across the sweep (typically 61 to 66% during the long runs of the job examined). Time to first token at the first request rate was not systematically higher than at later rates: the median ratio of the 5 req/s value to the 10 req/s value was 1.02 across the 25 input and output shapes with outputs up to 5,120 tokens. Two load points with 1,024-token inputs were exceptions (274 and 296 ms at 5 req/s against about 100 to 125 ms at later rates) and may be cold-cache cases. The effect on time to first token therefore appears small, but it was not isolated, and the effect on throughput was not examined. A repeat should disable prefix caching or vary the seed per load point.
- **No saturation rate is reported.** With 100-request load points and submission windows of 1 to 20 s, achieved throughput rises with rate for short outputs mainly because the requests arrive

sooner. A rate at which throughput stops improving would reflect the test design as much as the system.

- **Estimates.** The percent-of-peak figure and the maximum-throughput estimates in Section 6 rest on inputs that were not measured, the dense FP8 peak and the memory bandwidth, and they assume the PCIe links run at their theoretical peak (the link capability, PCIe 5.0 x16 per GPU, was confirmed). Halving the assumed FP8 peak would double the percent of peak and halve the compute ceilings and the ridge point. The ridge also varies inversely with memory bandwidth. At 80 to 85% link efficiency, the PCIe ceiling would be about 8,500 to 9,000 tokens/s instead of about 10,600. FLOPs per token (2 × parameters) ignores attention, which grows with context length, and KV capacity figures are upper bounds.
- **Scope.** A single model on a single hardware configuration, random prompts, and FP8 weights with a BF16 KV cache. Other precisions (BF16 weights, FP8 KV cache, FP4) were not tested.

Appendix A

A.1. Test Environment

Table A1. Test environment

Component	Detail
Server	Dell PowerEdge XE7745, two CPU sockets, 8 GPUs per node; 4 GPUs allocated to the job
GPUs	NVIDIA RTX PRO 6000 Blackwell Server Edition, 96 GB each; GPU indices 0 to 3 used
GPU interconnect	PCIe 5.0 x16 per GPU (maximum link generation and width confirmed with `nvidia-smi`); no NVLink. GPUs 0 to 3 share one CPU socket (NUMA node 0) and communicate through its PCIe host bridges; GPUs 4 to 7 are on the other socket
Software	Ubuntu 24.04.4 LTS; NVIDIA driver 590.48.01; CUDA 13.1; Python 3.12; vLLM 0.30.0 [6]
Scheduler	Slurm; one job per server instance, 4 GPUs (`--gres=gpu:4`), 32 CPUs, 256 GB memory
Storage	Model checkpoint on a network file system (read once at server start-up); Python environment, logs and results on node-local NVMe
Monitoring	`nvidia-smi` and NVIDIA DCGM [5]
Collection dates	25 to 26 September 2026

The Python environment was moved to node-local storage after early runs stalled while importing many small files from the network file system. Reading the large checkpoint from the network file system caused no such problem.

A.2. Workload and Serving

The model is the pre-quantized FP8 checkpoint of Llama 3.1 70B Instruct published by Red Hat AI [8], derived from Meta's model [7]. After tensor-parallel sharding, the weights occupy 16.98 GiB per GPU

(reported in the vLLM start-up log), and the log shows FP8 batched matrix-multiply kernels in use. The server was started with:

```
vllm serve <model-dir> \
  --tensor-parallel-size 4 \
  --gpu-memory-utilization 0.88 \
  --max-model-len 16384 \
  --served-model-name llama-3.1-70b-fp8
```

All other options were left at vLLM defaults. The server log reports a prefix-cache hit rate, so automatic prefix caching was active (A.4). No `--kv-cache-dtype` flag was set, so the KV cache used vLLM's default, which follows the model's compute type (BF16).

Load was generated by vLLM's built-in benchmark client (`vllm bench serve`), running on the same node and sending requests to the server's `/v1/completions` endpoint. One load point is one combination of input length, output length and request rate, with 100 requests.

Table A2. Workload parameters

Parameter	Setting
Prompts	Random token prompts (`--dataset-name random`), fixed seed 42
Nominal input lengths	128, 256, 384, 512, 1,024 tokens
Nominal output lengths	128, 512, 1,024, 2,048, 5,120, 10,240 tokens
Length variation	Uniform within ±30% of nominal (`--random-range-ratio 0.3`)
Request rates	5, 10, 20, 50, 100 req/s, Poisson arrivals (client default burstiness 1.0)
Requests per load point	100
Concurrency limit	None at the client (`--max-concurrency` unset)
Sampling	Server defaults; the client was not set to force greedy decoding
Grid	5 × 6 × 5 = 150 load points

```
vllm bench serve --backend vllm --base-url http://localhost:8000 \
  --model llama-3.1-70b-fp8 --tokenizer <model-dir> \
  --dataset-name random --random-input-len <IL> --random-output-len <OL> \
  --random-range-ratio 0.3 --seed 42 --num-prompts 100 --request-rate <R> \
  --save-result --result-filename <file>.json
```

The procedure for a sweep was:

1. The server job starts, and the driver script waits until the `/v1/models` endpoint responds.
2. One warm-up run is executed and discarded (20 prompts, 256 input and 256 output tokens, 5 req/s, seed 1) to absorb one-time costs such as graph capture and cold caches.
3. Load points run one at a time in the order input length (outer), output length, request rate (inner). For each, the monitors start, the client runs, the monitors stop, and a row is written to the manifest. The client first sends one single-prompt test request before timing begins.
4. A 15 s pause separates load points.

5. The driver skips load points already marked completed in the manifest, so an interrupted sweep can be resumed. The full grid takes on the order of 12 hours.

A.3. Measurement

Table A3. Data sources

Source	Records	Sampling	Notes
vLLM benchmark client	Time to first token, time per output token, throughput, request counts	Per run (JSON)	Throughput uses the client's own duration
`nvidia-smi`	GPU utilization, memory used, power draw per GPU	Every 2 s	Board power; averaged over samples and the four GPUs
DCGM (`dcgmi dmon`)	GR engine, SM, SM occupancy, tensor and DRAM activity; PCIe TX and RX	Every 2 s	Fields 1001 to 1005, 1009, 1010; PCIe in bytes/s per GPU
vLLM server log	KV cache in use (% of pool)	About every 10 s	Matched to each load point by time window
Wall clock	Run time (`elapsed_s`)	Per run	Whole client command, including start-up and teardown

`nvidia-smi` lists all eight GPUs on the node even inside a job, and the DCGM daemon runs outside the job's environment. The DCGM group was therefore built from the UUIDs of the GPUs listed in `CUDA_VISIBLE_DEVICES`, and `dcgmi` was run with that variable unset. Averages and peaks cover all samples in the run window and all four GPUs.

```
nvidia-smi --query-gpu=timestamp,index,utilization.gpu,memory.used,memory.total,power.draw
\
  --format=csv -l 2 -f <gpu-log>.csv
dcgmi group -c sweep_group          # then: dcgmi group -g <id> -a <GPU UUIDs>
dcgmi dmon -g <id> -e 1001,1002,1003,1004,1005,1009,1010 -d 2000 > <dcgm-log>.log
```

Each load point produces a JSON file (client), a CSV (`nvidia-smi`), a log (DCGM) and one row in `manifest.csv`. An analysis script merges these with the server-log KV usage into `full_summary.csv`, and `Post Study Calcs.xlsx` holds the derived quantities as formulas. The derived quantities are:

- **FLOPs per token** = 2 × parameters (about 140 GFLOP; attention excluded).
- **Concurrency** (Little's law) = throughput × time per output token.
- **Energy per token** = throughput ÷ (average power per GPU × 4).
- **PCIe traffic per token** = average TX (or RX) bytes per second ÷ throughput.
- **KV cache per token per GPU** = 2 × layers × KV heads × head dimension × bytes per element ÷ 4.

A.4. Known Issues and Validity

- **Single trial per load point.** There is no variance estimate. One load point (256-token inputs, 1,024-token outputs, 20 req/s) shows a transient dip in utilization and power, discussed in Section 8.

- **Prefix caching.** Automatic prefix caching was enabled and the benchmark reuses the same seeded prompts across request rates, so many prompt tokens were served from cache (reported hit rates up to 99%). This may understate prefill cost and input-length effects. The effect on time to first token appears small but was not isolated (see A.4).
- **Measurement window.** `elapsed_s` and the activity and power averages cover the whole client command, including a few seconds of start-up and teardown. The client's own duration, which the throughput figures use, excludes them. This dilutes the averages for runs shorter than about two minutes.
- **KV cache sampling.** vLLM logs KV cache use about every 10 s, so the shortest runs (13 to 31 s) have only one to three samples and their KV peaks are coarse.
- **Failed requests.** All 150 load points completed their 100 requests with zero failures. The manifest status records only the client's exit code, which is 0 even when requests fail, so the failed-request count in each JSON file was checked separately.
- **Hardware counters.** DCGM profiling fields return N/A to non-root users unless `kernel.perf_event_paranoid` is 1 or lower (it was set to 1 on the node). The first samples of each run are also N/A and are excluded from averages.
- **Shared node.** The node's other four GPUs and its CPUs were not reserved for the study, and contention from other jobs was not measured.
- **Issues corrected before the final dataset.** An early pass used `--max-model-len 8192`, which rejected requests for 10,240-token outputs (HTTP 400); the affected load points were deleted and rerun with 16384. An early manifest script read three DCGM peak columns one field off, so the analysis uses the fully parsed `dcgm_*` columns from the raw logs instead.

Data and Code Availability. All load-point data and most of the calculations behind the tables in this report are in `Post Study Calcs.xlsx`: a Data sheet with one row per load point (measured and derived columns), assumptions and derived constants, summaries by output length, input length and request rate, full output-length-by-rate grids, KV cache capacity, and the maximum-throughput estimates. Every derived value is a formula over the Data and Assumptions sheets. The workbook, the raw load-point files and the scripts used to run and analyze the sweep are available at @@repository link or DOI@@ under @@license@@.

References

1. W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang and I. Stoica. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP), 2023. https://arxiv.org/abs/2309.06180
2. G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim and B.-G. Chun. *Orca: A Distributed Serving System for Transformer-Based Generative Models*. 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22), 2022, pp. 521–538. https://www.usenix.org/conference/osdi22/presentation/yu

3. R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, J. Heek, K. Xiao, S. Agrawal and J. Dean. *Efficiently Scaling Transformer Inference.* Proceedings of Machine Learning and Systems 5 (MLSys 2023). https://arxiv.org/abs/2211.05102
4. A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. Gulavani, A. Tumanov and R. Ramjee. *Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve.* 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24), 2024. https://www.usenix.org/conference/osdi24/presentation/agrawal
5. NVIDIA. *DCGM Documentation: Profiling Metrics*. https://docs.nvidia.com/datacenter/dcgm/latest/learn/modules/profiling.html
6. vLLM Project. *vLLM Documentation* (engine arguments, including --gpu-memory-utilization; vllm bench serve; metrics). https://docs.vllm.ai — version [your vLLM version]
7. Meta AI. *Llama 3.1 Model Card.* https://github.com/meta-llama/llama-models/blob/main/models/llama3_1/MODEL_CARD.md